# PLATFORM-OAUTH

## Architecture de fonctionnement et dossier de recette Postman

### Authorization Code avec PKCE S256
### Refresh tokens, rotation et persistance

Version documentaire : 28 octobre 2026
Branche : feature/authorization-code-pkce-refresh-token-lifecycle

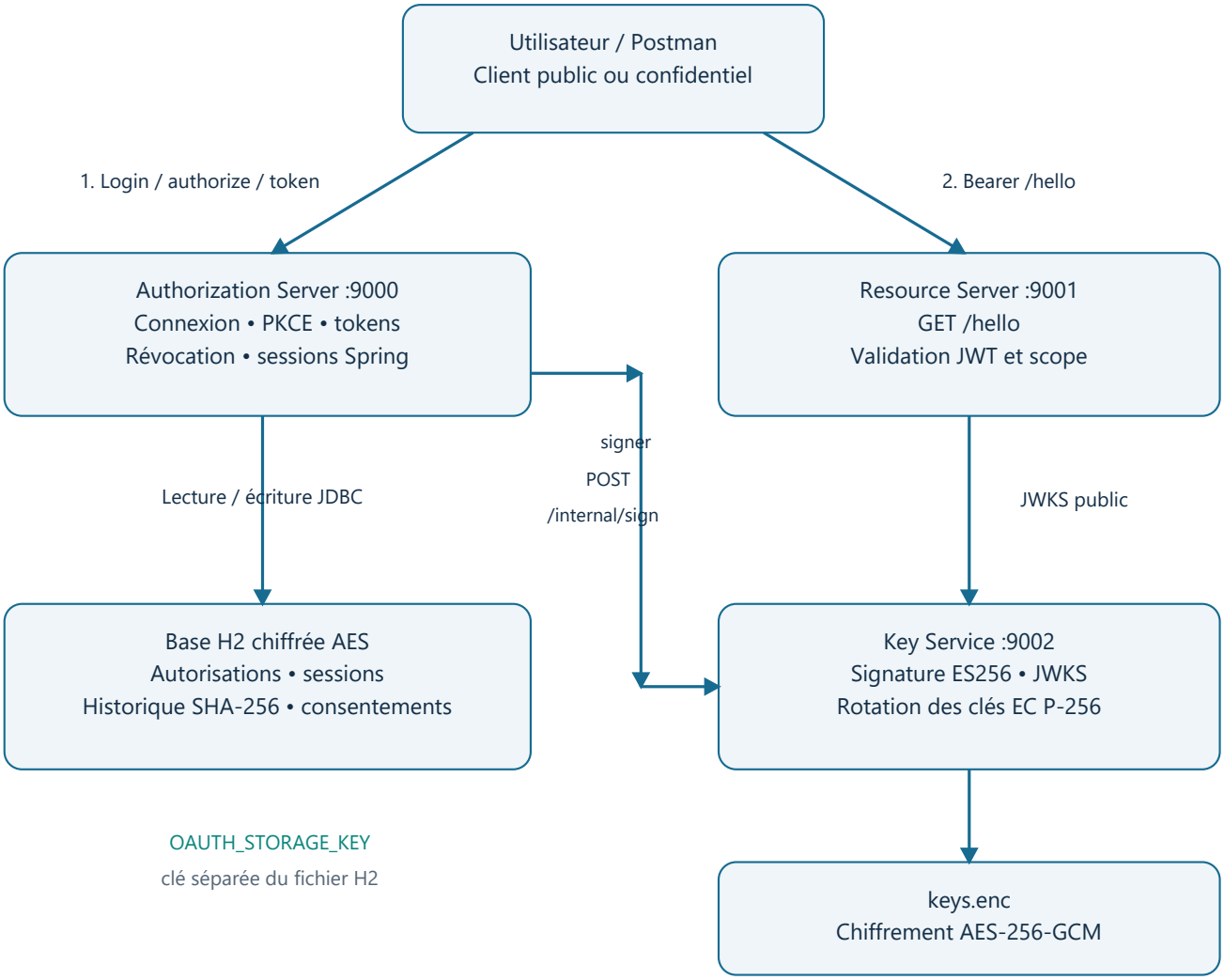
Document fondé sur le code courant et le plan manuel du dépôt. Les sorties des scénarios sont des résultats attendus : leur présence dans ce PDF ne signifie pas que la recette manuelle a été exécutée.

Les exemples utilisent uniquement des variables et des valeurs fictives. Aucune clé, aucun mot de passe ni token réel ne figure dans ce document.


| Lecture | Contenu |
|---|---|
| Partie 1 | Architecture, responsabilités, stockage et séquences |
| Partie 2 | Préparation et requêtes Postman R1 à R7 |
| Partie 3 | 56 fiches de scénario : entrées, sorties, explications |
| Annexes | Scripts Postman, limites de recette et traçabilité |


Navigation : les signets du lecteur PDF permettent d'ouvrir directement une section ou une fiche de scénario.

# 1. Architecture applicative




Le client obtient une autorisation auprès de l'AS, puis présente son access token au Resource Server. Le Key Service détient les clés privées et signe à la demande de l'AS. Le RS télécharge les clés publiques et vérifie les JWT localement.

Le RS consulte par défaut le JWKS du Key Service (:9002). L'AS expose également un proxy public GET /oauth2/jwks. Le RS ne consulte pas la base des refresh tokens à chaque appel.

Les flèches représentent les demandes ; les réponses reviennent sur les mêmes connexions. HTTP loopback correspond à la configuration locale. TLS et des URL d'issuer cohérentes doivent être configurés pour un déploiement.

# 2. Responsabilités et données

| Composant | Responsabilité / accès |
|---|---|
| Authorization Server :9000 | Authentifie les utilisateurs ; contrôle PKCE, client et scopes ; émet les tokens ; gère la révocation. Ne possède aucune clé privée de signature. |
| Resource Server :9001 | GET /hello. Vérifie ES256, kid, signature, issuer, audience, exp, nbf, iat, sub, jti et scope hello.read. |
| Key Service :9002 | Génère P-256 ; signe avec le compte signer ; rotation réservée au compte admin ; publie seulement les clés publiques. |
| H2 / authorization-data | Autorisations, codes et tokens courants, sessions Spring, consentements et historique des refresh tokens. Fichier chiffré AES. |
| keys.enc / signing-keys | Clé privée active et anciennes clés publiques retenues ; chiffrement AES-256-GCM avec une autre clé maîtresse. |

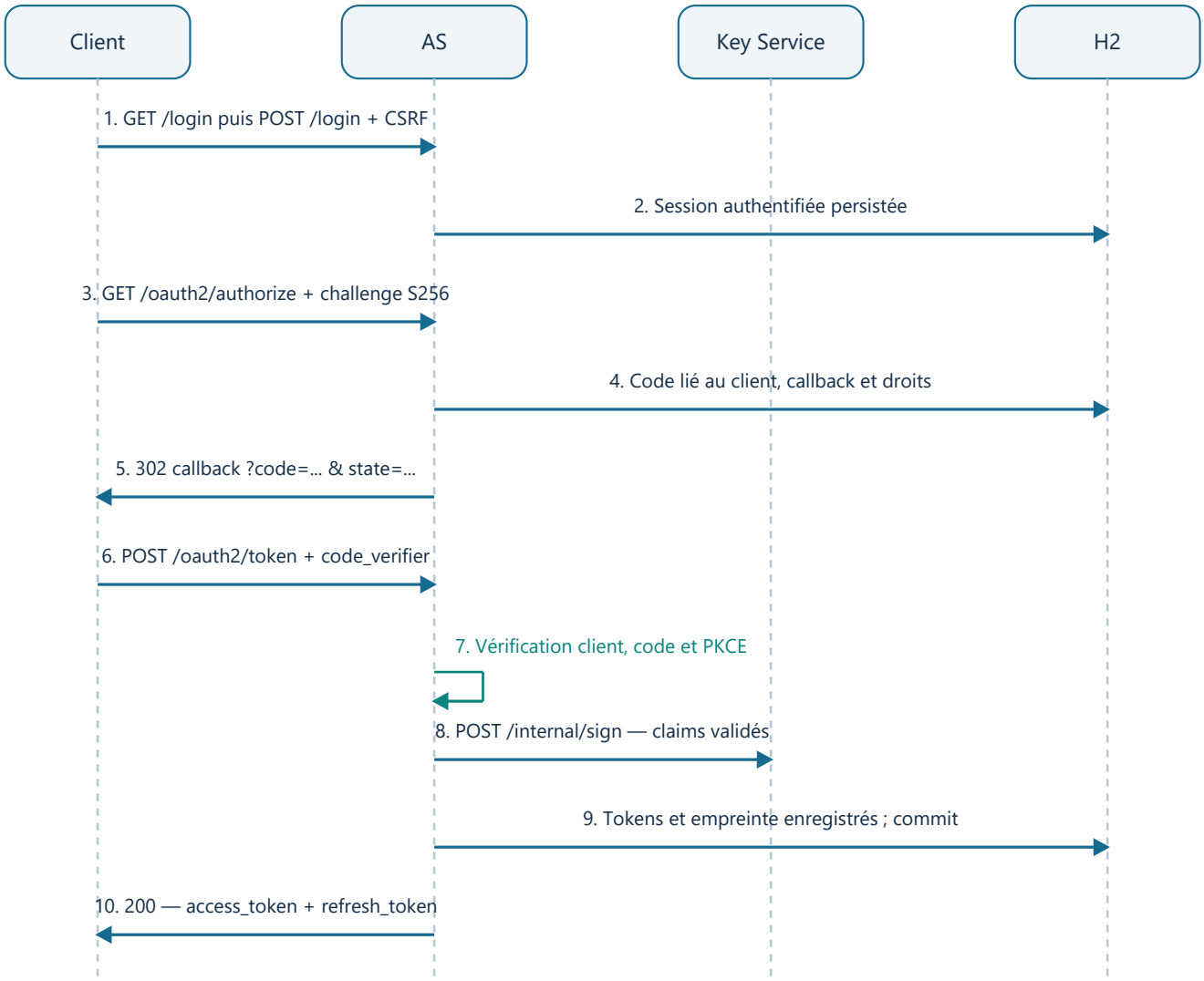
## Deux stockages et deux clés indépendantes

OAUTH_STORAGE_KEY protège la base de l'AS. KEY_SERVICE_MASTER_KEY protège keys.enc. Le secret de signature sert à authentifier l'AS auprès du Key Service : il n'est pas la clé privée ES256. Les clés doivent rester séparées des fichiers sauvegardés.

Les utilisateurs et les clients de démonstration restent configurés en mémoire au démarrage, avec des identifiants stables. Les autorisations et sessions sont persistantes. Les deux clients sont internes et ne déclenchent pas d'écran de consentement ; le service JDBC de consentement est néanmoins disponible.

| Objet | Durée / règle |
|---|---|
| Code d'autorisation | 2 minutes ; usage unique |
| Access token JWT | 5 minutes ; scope hello.read ; audience portage-api |
| Refresh token | 8 heures depuis son émission ; remplacé à chaque renouvellement |
| Famille | 7 jours au maximum depuis la première émission |
| Session navigateur | 30 minutes d'inactivité ; distincte de l'autorisation OAuth |

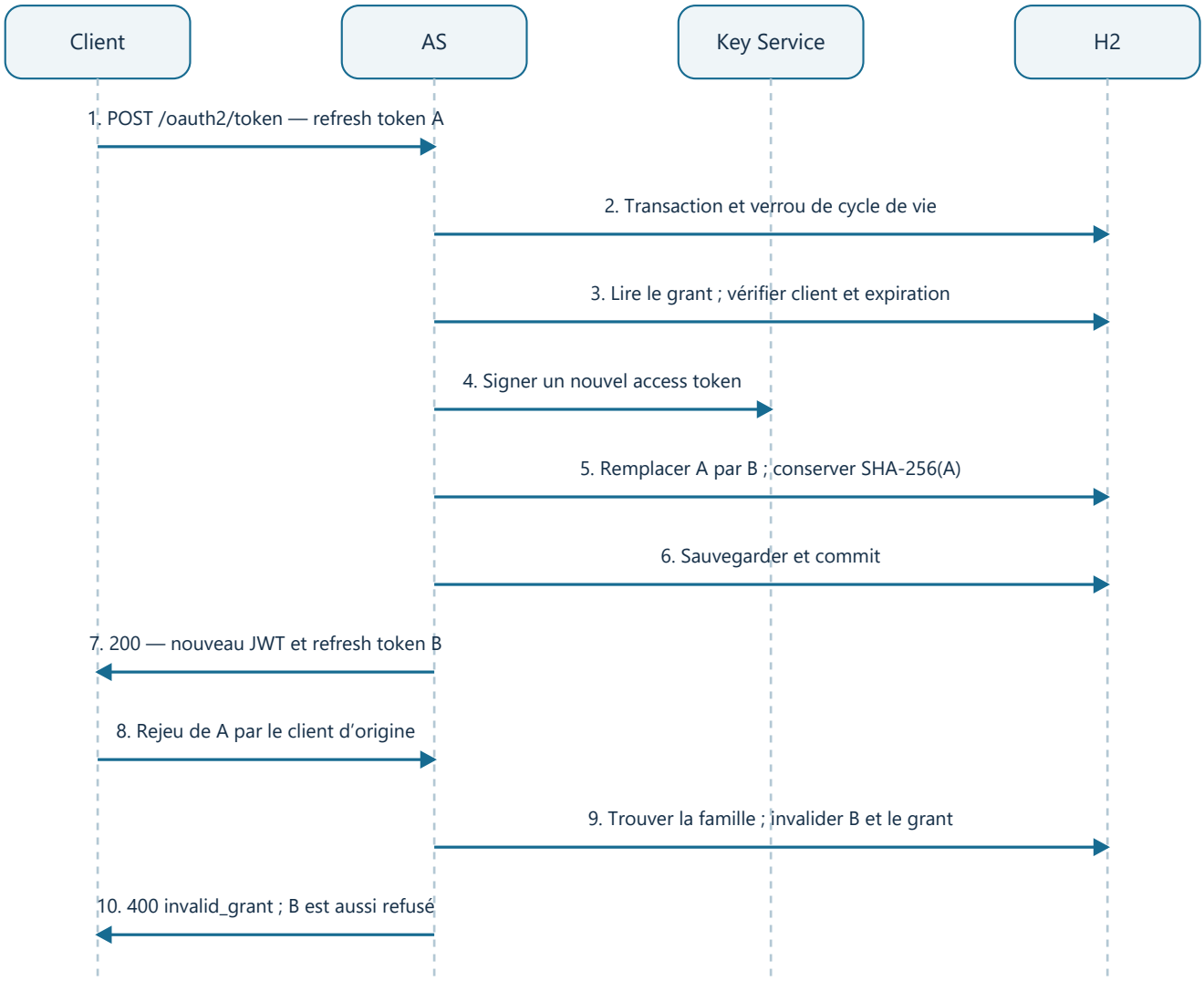
# 3. Séquence : connexion et émission




Le code_verifier n'est pas transmis à /authorize : seul son SHA-256 encodé en Base64URL sans padding est envoyé. Le client garde le vérificateur et le présente au token endpoint.

Le client vérifie que state reçu est identique à state envoyé. Spring protège le code et PKCE ; la vérification de state au callback reste une responsabilité du client.

# 4. Séquence : renouvellement et rejeu




Un autre client présentant A est refusé sans révoquer la famille du propriétaire. Les demandes de token et de révocation sont sérialisées par un verrou JDBC global. La réponse portant B est envoyée après commit.

Deux usages concurrents du même refresh token conduisent à une réussite et un refus. La seconde utilisation est un rejeu : le refresh token issu de la première est alors lui aussi invalidé. Le client doit sérialiser ses renouvellements.

# 5. Révocation, ressources et limites

## Révocation explicite

```
POST /oauth2/revoke
client_id=portage-public
token={{refresh_token}}
token_type_hint=refresh_token

HTTP 200 ; puis renouvellement : HTTP 400 {"error":"invalid_grant"}
```

La révocation d'un token inconnu retourne aussi 200. Révoquer un ancien refresh token de sa propre famille invalide la famille active. Les JWT déjà signés restent acceptables jusqu'à expiration : le RS ne fait pas d'introspection à chaque appel.

## Signature et rotation des clés

La rotation des clés ES256 est distincte de la rotation des refresh tokens. Le Key Service effectue une rotation quotidienne et une rotation manuelle par POST /internal/rotate avec le compte admin. Les anciennes clés publiques restent publiées 900 secondes par défaut pour vérifier les JWT encore valables.

## Limites actuelles

• H2 embarqué et le Key Service sont prévus pour une instance locale. Ne pas partager un fichier H2 entre conteneurs ; une architecture distribuée demanderait une base partagée et des verrous adaptés.

• Les empreintes des refresh tokens sont conservées ; la purge des familles expirées n'est pas automatisée. La rotation des clés maîtresses de chiffrement exige une procédure de migration.

• La déconnexion de la session navigateur ne révoque pas implicitement les autorisations OAuth. Une nouvelle connexion et une nouvelle autorisation créent une famille indépendante.

• Les durées des tokens sont codées dans les classes ; aucune variable d'environnement ne permet actuellement de les raccourcir.

• La gestion 503 temporarily_unavailable est explicitement câblée pour les erreurs OAuth de ce type. Un JwtEncodingException sur le nouveau flux Spring ne doit pas être supposé produire automatiquement 503 : le comportement de panne du signataire nécessite sa propre vérification.

# 6. Classes et points d’extension

| Classe / fichier | Rôle |
|---|---|
| AuthorizationSecurityConfiguration | Chaînes de sécurité, utilisateurs de connexion, deux clients, S256, générateur JWT et endpoints. Classe existante modifiée. |
| AuthorizationPersistenceConfiguration | Source H2 chiffrée, schémas JDBC Spring, sessions et cookies. |
| PublicRefreshClientAuthentication | Identification NONE pour refresh et revoke ; laisse l’échange de code et PKCE aux composants Spring. |
| RotatingAuthorizationService | Délègue la persistance à JdbcOAuth2AuthorizationService ; conserve les empreintes et invalide les familles rejouées. |
| RotatingRefreshTokenGenerator | Refresh token aléatoire ; plafond de famille de sept jours. |
| TokenTransactionFilter | Verrou et transaction englobant token/revoke ; tampon de réponse avant commit. |
| authorization-lifecycle.sql | Tables oauth_lifecycle_lock et oauth_refresh_history. |
| RemoteSigningConfiguration | JwtEncoder HTTP vers le Key Service. Aucun stockage de clé privée dans l’AS. |
| Tests ajoutés | AuthorizationCodePkceHttpTest (10 tests HTTP) et AuthorizationPersistenceTest (2 tests de stockage). |

Les données Spring d’autorisation associent registeredClientId, principalName, authorizedScopes, attributs, code, access token et refresh token courant. L’historique ajoute token_hash, authorization_id, client_id et expires_at. Les anciennes valeurs brutes des refresh tokens ne sont pas conservées dans cette table d’historique.

# 7. Préparer la recette Postman

## Démarrage

Construire les JAR avec mvn clean verify, puis lancer les trois services dans trois terminaux héritant des variables requises. Ne pas utiliser scripts/demo.py pour une séance persistante : il arrête ses processus à la fin et active le grant historique.

```
java -jar key-service/target/key-service-1.0.0-SNAPSHOT-exec.jar
java -jar authorization-server/target/authorization-server-1.0.0-SNAPSHOT.jar
java -jar resource-server/target/resource-server-1.0.0-SNAPSHOT.jar
```

| Variables serveur | Usage |
|---|---|
| OAUTH_USER1_PASSWORD … OAUTH_USER5_PASSWORD | Cinq mots de passe distincts ; user1 à user5. |
| OAUTH_CLIENT_SECRET | Secret de demo-client. |
| KEY_SERVICE_SIGNING_SECRET / ADMIN_SECRET | Secrets distincts, au moins 32 caractères ; noms complets préfixés KEY_SERVICE_. |
| KEY_SERVICE_MASTER_KEY | Base64 de 32 octets aléatoires ; Key Service uniquement. |
| OAUTH_STORAGE_KEY | Autre Base64 de 32 octets aléatoires ; AS uniquement. |
| OAUTH_STORAGE_DIRECTORY | Répertoire persistant de test. |
| OAUTH_REDIRECT_URI | http://127.0.0.1:8080/callback |
| OAUTH_SECURE_COOKIE | false pour la recette HTTP locale ; true par défaut. |
| OAUTH_LEGACY_GRANT_ENABLED | false pour la recette PKCE. |

Avec Docker Compose, conserver le volume authorization-data et le volume signing-keys. Le compose courant ne transmet pas OAUTH_SECURE_COOKIE : pour une recette HTTP avec Docker, ajouter explicitement ce réglage dans une surcharge locale, ou utiliser HTTPS. Ne pas effacer les volumes pour un scénario de redémarrage.

# 8. Variables et conventions de test

| Variable Postman | Valeur |
|---|---|
| as / rs | http://127.0.0.1:9000 / http://127.0.0.1:9001 |
| client_id | portage-public ; refaire les cas nominaux avec demo-client |
| client_secret | Valeur locale de OAUTH_CLIENT_SECRET |
| username / password | user1 / valeur locale de OAUTH_USER1_PASSWORD |
| redirect_uri | http://127.0.0.1:8080/callback |
| code_verifier | dBjftJeZ4CVP-mB92K27uhbUJU1p1r_wW1gFWFOEjXk |
| code_challenge | E9Melhoa2OwvFrEMTJguCHaoeK1t8URWbuGJSstw-cM |
| Variables capturées | csrf, state, authorization_code, access_token, refresh_token, previous_refresh_token |

Cette paire PKCE est un vecteur public RFC 7636, utilisable uniquement pour une recette reproductible. Un vrai client génère un nouveau vérificateur aléatoire par demande d'autorisation.

## Réglages

• Désactiver Automatically follow redirects pour voir les 302 et leur Location.

• Garder le cookie jar actif. Utiliser 127.0.0.1 partout, sans alterner avec localhost.

• POST : Body > x-www-form-urlencoded. Client public : No Auth. Client confidentiel : Basic Auth uniquement sur token/revoke.

• Le callback n'a pas besoin de serveur : Postman ne suit pas la redirection et lit Location.

• Chaque test destructeur repart d'une nouvelle autorisation. A, B et C désignent des refresh tokens fictifs, jamais leurs valeurs réelles.

• Une sortie « refus OAuth » signifie 4xx ou redirection OAuth avec error sur /authorize, sans aucun token/code de succès. Ne pas exiger un corps JSON sur une redirection.

# 9.1 Requête R1 — Formulaire

## Entrée

```
GET {{as}}/login
Authorization: No Auth
```

## Sortie attendue

200 text/html ; formulaire avec champ _csrf ; cookie SESSION.

## Explication

Le cookie lie la requête de connexion à sa session. Le champ CSRF est capturé depuis le formulaire, sans valeur inventée.

# 9.2 Requête R2 — Connexion

## Entrée

```
POST {{as}}/login
Authorization: No Auth ; cookie de R1
username={{username}}
password={{password}}
_csrf={{csrf}}
```

## Sortie attendue

302 ; Location sans ?error ; session authentifiée. Ne pas suivre la redirection vers /.

## Explication

Le mot de passe est vérifié par Spring avec le hash configuré. Le principal authentifié utilise le sujet stable subject-N.

# 9.3 Requête R3 — Code PKCE

## Entrée

```
GET {{as}}/oauth2/authorize
Accept: text/html ; cookie authentifié
response_type=code
client_id=portage-public
redirect_uri={{redirect_uri}}
scope=hello.read
state={{state}}
code_challenge={{code_challenge}}
code_challenge_method=S256
```

## Sortie attendue

302 ; Location: {{redirect_uri}}?code=<CODE>&state=<STATE>. Vérifier state puis capturer code.

## Explication

Le code est lié au client, au callback, aux droits et au challenge. Le code n'est pas un access token ; il ne permet pas directement /hello.

Préparer state avec {{$guid}} et vérifier sa correspondance au retour. Le script complet de capture figure en annexe. Un code non échangé expire au bout de deux minutes.

# 9.4 Requête R4 — Échange de code

## Entrée

```
POST {{as}}/oauth2/token
Authorization: No Auth
grant_type=authorization_code
client_id=portage-public
code={{authorization_code}}
redirect_uri={{redirect_uri}}
code_verifier={{code_verifier}}
```

## Sortie attendue

200 JSON :
{"access_token":"<JWT>","refresh_token":"<RT_A>","token_type":"Bearer","expires_in":299,"scope":"hello.read"}.
expires_in proche de 300 ; Cache-Control: no-store.

## Explication

Spring vérifie le code, son expiration, le callback, le client et la preuve PKCE. Le JWT est signé à distance puis les tokens sont persistés.

Variante confidentielle : remplacer client_id par demo-client et choisir Basic Auth, username demo-client, password {{client_secret}}. Ne pas ajouter de client_secret dans le formulaire. PKCE reste obligatoire à R4.

# 9.5 Requête R5 — Ressource

## Entrée

```
GET {{rs}}/hello
Authorization: Bearer {{access_token}}
```

## Sortie attendue

200 JSON : {"message":"HelloWorld","subject":"subject-1"}.

## Explication

Le RS vérifie les clés publiques, la signature, les claims et le scope. Cette étape prouve plus qu'un simple décodage Base64 du JWT.

# 9.6 Requête R6 — Renouvellement

## Entrée

```
POST {{as}}/oauth2/token
Authorization: No Auth
grant_type=refresh_token
client_id=portage-public
refresh_token={{refresh_token}}
```

## Sortie attendue

200 JSON : nouveau access_token et refresh_token B différent de A ; scope hello.read. Même sub et aud.

## Explication

La rotation rend A inutilisable ; le client doit enregistrer B. Ne jamais déclencher plusieurs renouvellements avec la même valeur.

Variante confidentielle : remplacer client_id par demo-client et choisir Basic Auth, username demo-client, password {{client_secret}}. Ne pas ajouter de client_secret dans le formulaire. PKCE reste obligatoire à R4.

Avant Send, copier refresh_token dans previous_refresh_token. Après 200 seulement, mettre à jour access_token et refresh_token. Ne pas utiliser ce script de capture sur les variantes négatives.

# 9.7 Requête R7 — Révocation

## Entrée

```
POST {{as}}/oauth2/revoke
Authorization: No Auth
client_id=portage-public
token={{refresh_token}}
token_type_hint=refresh_token
```

## Sortie attendue

200, corps normalement vide. Une demande R6 avec le token révoqué retourne 400 {"error":"invalid_grant"}.

## Explication

La révocation concerne l'autorisation serveur. Le JWT autonome déjà remis au client reste utilisable jusqu'à son expiration.

Variante confidentielle : remplacer client_id par demo-client et choisir Basic Auth, username demo-client, password {{client_secret}}. Ne pas ajouter de client_secret dans le formulaire. PKCE reste obligatoire à R4.

# 10. Catalogue des scénarios

Chaque fiche décrit des entrées de test, des sorties attendues et le mécanisme vérifié. Les identifiants A/C/F/S/P correspondent au plan Markdown du dépôt ; CON01 complète le cas de concurrence.

| Famille | Identifiants | Nombre |
|---|---|---|
| Connexion / autorisation | A01 à A11 | 11 |
| Code / ressource | C01 à C12 | 12 |
| Refresh et liaison | F01 à F10 | 10 |
| Rejeu et révocation | S01 à S10 | 10 |
| Concurrence | CON01 | 1 |
| Persistance et configuration | P01 à P12 | 12 |

## Préconditions communes

Services démarrés, variables locales préparées, cookie de connexion valide quand requis. Toutes les demandes token/revoke utilisent x-www-form-urlencoded. Les tests nominaux et ceux d'authentification du client sont à refaire pour le client confidentiel avec HTTP Basic.

Les tests de rejeu, de code réutilisé et de révocation modifient durablement leur famille. Créer une famille neuve pour chaque scénario indépendant. Ne pas laisser les scripts de capture écraser le token de référence après une erreur.

## Différence entre preuve et résultat attendu

L'implémentation a précédemment passé 42 tests automatisés, dont 12 nouveaux. Le catalogue manuel élargit les contrôles et décrit des attentes à confirmer. Aucun résultat manuel n'est inventé dans ce document ; noter le statut réellement obtenu pour chaque essai.

## A01 — R1 → R2 avec les bons identifiants.

INPUT / PRÉPARATION

```
POST {{as}}/login
Authorization: No Auth ; cookie de R1
username={{username}}
password={{password}}
_csrf={{csrf}}

Manipulation : Exécuter R1 puis ce POST avec les valeurs valides.
```

OUTPUT ATTENDU

Connexion 302 sans `error`.

EXPLICATION

Le cookie et le CSRF associent la connexion à sa session ; les identifiants valides créent le principal authentifié.

## A02 — R2 avec mauvais mot de passe ou utilisateur inconnu, CSRF valide.

INPUT / PRÉPARATION

```
POST {{as}}/login
Authorization: No Auth ; cookie de R1
username={{username}}
password={{password}}
_csrf={{csrf}}

Manipulation : Essai 1 : password=incorrect ; essai 2 : username=inconnu. Conserver un CSRF
valide.
```

OUTPUT ATTENDU

Redirection vers `/login?error`, aucune session authentifiée.

EXPLICATION

La connexion doit refuser les identités invalides sans émettre de code ou de token. Reprendre R1 si le CSRF change.

## A03 — R2 sans `_csrf` ou avec valeur incorrecte.

INPUT / PRÉPARATION

```
POST {{as}}/login
Authorization: No Auth ; cookie de R1
username={{username}}
password={{password}}
_csrf={{csrf}}

Manipulation : Essai 1 : retirer _csrf ; essai 2 : _csrf=incorrect.
```

OUTPUT ATTENDU

403, connexion refusée.

EXPLICATION

Le refus CSRF intervient avant la création d'une session authentifiée, même avec un mot de passe correct.

## A04 — Effacer le cookie de session puis appeler R3 avec Accept HTML.

INPUT / PRÉPARATION

```
GET {{as}}/oauth2/authorize
Accept: text/html ; cookie authentifié
response_type=code
client_id=portage-public
redirect_uri={{redirect_uri}}
scope=hello.read
state={{state}}
code_challenge={{code_challenge}}
code_challenge_method=S256

Manipulation : Supprimer le cookie SESSION du domaine 127.0.0.1 avant Send.
```

OUTPUT ATTENDU

Redirection vers `/login`, aucun code.

EXPLICATION

Un client OAuth n'authentifie pas l'utilisateur : sans session, l'AS exige la connexion.

## A05 — R3 sans `code_challenge` et sans méthode.

INPUT / PRÉPARATION

```
GET {{as}}/oauth2/authorize
Accept: text/html ; cookie authentifié
response_type=code
client_id=portage-public
redirect_uri={{redirect_uri}}
scope=hello.read
state={{state}}
code_challenge={{code_challenge}}
code_challenge_method=S256

Manipulation : Retirer code_challenge et code_challenge_method.
```

OUTPUT ATTENDU

Refus OAuth, aucun code.

EXPLICATION

La configuration requireProofKey et la validation S256 empêchent un échange de code sans preuve PKCE.

## A06 — R3 avec méthode `plain`, puis avec méthode inconnue.

INPUT / PRÉPARATION

```
GET {{as}}/oauth2/authorize
Accept: text/html ; cookie authentifié
response_type=code
client_id=portage-public
redirect_uri={{redirect_uri}}
scope=hello.read
state={{state}}
code_challenge={{code_challenge}}
code_challenge_method=S256

Manipulation : Essai 1 : code_challenge_method=plain ; essai 2 :
code_challenge_method=inconnue.
```

OUTPUT ATTENDU

Refus OAuth, aucun code.

EXPLICATION

Seule S256 est autorisée. La méthode plain ne protège pas le secret de vérification sur la demande d'autorisation.

## A07 — R3 avec challenge trop court ou caractères hors Base64URL.

INPUT / PRÉPARATION

```
GET {{as}}/oauth2/authorize
Accept: text/html ; cookie authentifié
response_type=code
client_id=portage-public
redirect_uri={{redirect_uri}}
scope=hello.read
state={{state}}
code_challenge={{code_challenge}}
code_challenge_method=S256

Manipulation : Essai 1 : code_challenge=abc ; essai 2 : challenge avec caractères + ou /.
```

OUTPUT ATTENDU

Refus OAuth, aucun code.

EXPLICATION

Le challenge S256 attendu comporte 43 caractères Base64URL sans padding. Un challenge invalide ne doit pas produire de code.

## A08 — R3 avec scope `admin`.

INPUT / PRÉPARATION

```
GET {{as}}/oauth2/authorize
Accept: text/html ; cookie authentifié
response_type=code
client_id=portage-public
redirect_uri={{redirect_uri}}
scope=hello.read
state={{state}}
code_challenge={{code_challenge}}
code_challenge_method=S256

Manipulation : Remplacer scope par admin.
```

OUTPUT ATTENDU

`invalid_scope`, aucun code.

EXPLICATION

Le client n'est enregistré que pour hello.read ; un scope supplémentaire ne peut pas être auto-accordé.

## A09 — R3 avec `client_id` inconnu ou callback non enregistré.

INPUT / PRÉPARATION

```
GET {{as}}/oauth2/authorize
Accept: text/html ; cookie authentifié
response_type=code
client_id=portage-public
redirect_uri={{redirect_uri}}
scope=hello.read
state={{state}}
code_challenge={{code_challenge}}
code_challenge_method=S256

Manipulation : Essai 1 : client_id=inconnu ; essai 2 :
redirect_uri=http://127.0.0.1:8080/autre.
```

OUTPUT ATTENDU

Refus ; ne pas rediriger vers un callback non validé.

EXPLICATION

Le serveur doit d'abord valider le client et le callback. Il ne doit pas envoyer une erreur contenant des données d'autorisation vers une URI non enregistrée.

## A10 — R3 nominal, comparer state envoyé et state reçu.

INPUT / PRÉPARATION

```
GET {{as}}/oauth2/authorize
Accept: text/html ; cookie authentifié
response_type=code
client_id=portage-public
redirect_uri={{redirect_uri}}
scope=hello.read
state={{state}}
code_challenge={{code_challenge}}
code_challenge_method=S256

Manipulation : Envoyer state=<GUID_FRAIS> et comparer au state dans Location.
```

OUTPUT ATTENDU

Valeurs identiques. La comparaison est à la charge du client.

EXPLICATION

L'AS renvoie state ; le client doit effectuer la comparaison. Un state modifié doit être rejeté par le client, pas supposé vérifié par l'AS au callback.

## A11 — R3 pour `user1`, puis nouvelle connexion distincte pour `user2`.

INPUT / PRÉPARATION

```
POST {{as}}/oauth2/token
Authorization: No Auth
grant_type=authorization_code
client_id=portage-public
code={{authorization_code}}
redirect_uri={{redirect_uri}}
code_verifier={{code_verifier}}

Manipulation : R1/R2 avec user1, puis R3/R4. Effacer les cookies ; refaire avec user2 et son
mot de passe.
```

OUTPUT ATTENDU

JWT avec `subject-1`, puis `subject-2` ; pas de mélange des sessions.

EXPLICATION

Le sujet vient du compte authentifié, jamais d'un paramètre subject libre dans le flux PKCE. Comparer les deux JWT et les réponses de /hello.

## C01 — R4 nominal pour les deux clients.

INPUT / PRÉPARATION

```
POST {{as}}/oauth2/token
Authorization: No Auth
grant_type=authorization_code
client_id=portage-public
code={{authorization_code}}
redirect_uri={{redirect_uri}}
code_verifier={{code_verifier}}

Manipulation : Exécuter une fois pour portage-public ; nouvelle autorisation pour demo-client
 + Basic valide.
```

OUTPUT ATTENDU

200 avec access et refresh tokens.

EXPLICATION

Les deux types de clients doivent produire un access et un refresh token, avec S256 obligatoire dans les deux cas.

## C02 — R4 avec mauvais `code_verifier`, puis sans vérificateur.

INPUT / PRÉPARATION

```
POST {{as}}/oauth2/token
Authorization: No Auth
grant_type=authorization_code
client_id=portage-public
code={{authorization_code}}
redirect_uri={{redirect_uri}}
code_verifier={{code_verifier}}

Manipulation : Code frais par essai :
code_verifier=bbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbb ; puis retirer code_verifier.
```

OUTPUT ATTENDU

Refus OAuth 400/401, aucun token.

EXPLICATION

Le vérificateur doit correspondre au challenge enregistré. Selon la phase de rejet Spring, le statut est 400 ou 401 ; aucun token ne doit être retourné.

## C03 — Mauvais vérificateur et paramètre `portage.public-refresh=true`.

INPUT / PRÉPARATION

```
POST {{as}}/oauth2/token
Authorization: No Auth
grant_type=authorization_code
client_id=portage-public
code={{authorization_code}}
redirect_uri={{redirect_uri}}
code_verifier={{code_verifier}}

Manipulation : code_verifier=bbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbb ; ajouter
portage.public-refresh=true.
```

OUTPUT ATTENDU

Même refus : le paramètre ne contourne pas PKCE.

EXPLICATION

Une chaîne HTTP ne peut pas forger le marqueur booléen interne du provider réservé au refresh/revoke. Le mauvais PKCE doit rester refusé.

## C04 — R4 avec un autre `redirect_uri`.

INPUT / PRÉPARATION

```
POST {{as}}/oauth2/token
Authorization: No Auth
grant_type=authorization_code
client_id=portage-public
code={{authorization_code}}
redirect_uri={{redirect_uri}}
code_verifier={{code_verifier}}

Manipulation : redirect_uri=http://127.0.0.1:8080/autre ; garder le code et le vérificateur
corrects.
```

OUTPUT ATTENDU

400 / invalid_grant.

EXPLICATION

Le callback de l'échange doit correspondre à celui de la demande initiale : un code intercepté ne doit pas être échangé avec une autre URI.

## C05 — R4 avec un code aléatoire.

INPUT / PRÉPARATION

```
POST {{as}}/oauth2/token
Authorization: No Auth
grant_type=authorization_code
client_id=portage-public
code={{authorization_code}}
redirect_uri={{redirect_uri}}
code_verifier={{code_verifier}}

Manipulation : code=code-invente ; garder un vérificateur conforme.
```

OUTPUT ATTENDU

Refus OAuth, aucun token.

EXPLICATION

Aucune autorisation persistée ne correspond à cette valeur ; le serveur ne doit jamais générer de tokens à partir d'un code arbitraire.

## C06 — Obtenir un code, attendre plus de deux minutes, R4.

INPUT / PRÉPARATION

```
POST {{as}}/oauth2/token
Authorization: No Auth
grant_type=authorization_code
client_id=portage-public
code={{authorization_code}}
redirect_uri={{redirect_uri}}
code_verifier={{code_verifier}}

Manipulation : Créer le code par R3 ; attendre au moins 2 min 10 s sans l’échanger ; envoyer
R4.
```

OUTPUT ATTENDU

400 / invalid_grant.

EXPLICATION

Le code est volontairement très court. L’attente commence à sa création, pas à l’ouverture de Postman.

## C07 — R4 nominal, puis rejouer exactement le même code.

INPUT / PRÉPARATION

```
POST {{as}}/oauth2/token
Authorization: No Auth
grant_type=authorization_code
client_id=portage-public
code={{authorization_code}}
redirect_uri={{redirect_uri}}
code_verifier={{code_verifier}}

Manipulation : Envoyer une première fois R4, conserver le refresh A ; renvoyer exactement le
même code ; tenter R6 avec A.
```

OUTPUT ATTENDU

400 / invalid_grant ; le refresh token issu du premier échange est invalidé.

EXPLICATION

Un code à usage unique rejoué invalide également les tokens associés via le provider Spring. Le premier JWT peut cependant rester accepté jusqu’à expiration.

## C08 — Utiliser le code avec un autre client authentifié.

INPUT / PRÉPARATION

```
POST {{as}}/oauth2/token
Authorization: No Auth
grant_type=authorization_code
client_id=portage-public
code={{authorization_code}}
redirect_uri={{redirect_uri}}
code_verifier={{code_verifier}}

Manipulation : Obtenir un code pour portage-public ; échanger avec client_id=demo-client et
son Basic valide.
```

OUTPUT ATTENDU

Refus OAuth, aucun token ; reprendre ensuite avec une nouvelle autorisation.

EXPLICATION

Le code appartient à son client initial. Utiliser des identifiants valides d'un autre client ne transfère pas cette autorisation.

## C09 — Client confidentiel : secret faux, puis authentification Basic absente.

INPUT / PRÉPARATION

```
POST {{as}}/oauth2/token
Authorization: No Auth
grant_type=authorization_code
client_id=portage-public
code={{authorization_code}}
redirect_uri={{redirect_uri}}
code_verifier={{code_verifier}}

Manipulation : Code pour demo-client ; essai Basic avec secret faux ; autre essai No Auth
avec client_id=demo-client.
```

OUTPUT ATTENDU

Refus d'authentification client, généralement 401 / invalid_client.

EXPLICATION

Un client confidentiel ne peut pas se présenter comme un client public. L'authentification du client doit échouer avant émission.

## C10 — R5 sans token, avec token modifié, puis token expiré après cinq minutes.

INPUT / PRÉPARATION

```
GET {{rs}}/hello
Authorization: Bearer {{access_token}}

Manipulation : Essai 1 : supprimer Authorization ; essai 2 : modifier le JWT ; essai 3 :
attendre plus de 5 min après iat.
```

OUTPUT ATTENDU

401 dans les trois cas.

EXPLICATION

Le RS distingue les tokens absents ou invalides des tokens vérifiables. Il ne fait pas confiance à une simple chaîne ayant une forme JWT.

## C11 — R5 avec le JWT nominal avant expiration.

INPUT / PRÉPARATION

```
GET {{rs}}/hello
Authorization: Bearer {{access_token}}

Manipulation : Utiliser le JWT nominal, non modifié et encore valable.
```

OUTPUT ATTENDU

200 / HelloWorld.

EXPLICATION

L'acceptation établit que la signature distante, les clés publiques, l'audience et le scope sont compatibles entre les services.

## C12 — R4 avec grant inconnu ou paramètre obligatoire absent.

INPUT / PRÉPARATION

```
POST {{as}}/oauth2/token
Authorization: No Auth
grant_type=authorization_code
client_id=portage-public
code={{authorization_code}}
redirect_uri={{redirect_uri}}
code_verifier={{code_verifier}}

Manipulation : Essai 1 : grant_type=inconnu ; autres essais : retirer code ou redirect_uri.
```

OUTPUT ATTENDU

Erreur OAuth 4xx, aucun token, jamais 200.

EXPLICATION

Les champs nécessaires au grant sont contrôlés. Le code exact de l'erreur dépend du champ et de la phase de validation ; consigner la réponse réellement obtenue.

## F01 — R6 nominal pour les deux clients.

INPUT / PRÉPARATION

```
POST {{as}}/oauth2/token
Authorization: No Auth
grant_type=refresh_token
client_id=portage-public
refresh_token={{refresh_token}}

Manipulation : Refaire avec chaque client et sa propre famille. Pour demo-client, ajouter son
 Basic valide.
```

OUTPUT ATTENDU

200 et refresh token différent.

EXPLICATION

Le nouveau refresh token doit différer du précédent. Le JWT renouvelé doit être accepté par le RS.

## F02 — Plusieurs R6 successifs, en utilisant toujours le dernier token.

INPUT / PRÉPARATION

```
POST {{as}}/oauth2/token
Authorization: No Auth
grant_type=refresh_token
client_id=portage-public
refresh_token={{refresh_token}}

Manipulation : Enchaîner trois renouvellements A -> B -> C -> D, en remplaçant la variable
après chaque 200.
```

OUTPUT ATTENDU

Toutes les demandes réussissent et tournent le token.

EXPLICATION

Une rotation normale ne doit pas révoquer la famille. Seule la valeur la plus récente est présentée à chaque fois.

## F03 — R6 avec `scope=hello.read`.

INPUT / PRÉPARATION

```
POST {{as}}/oauth2/token
Authorization: No Auth
grant_type=refresh_token
client_id=portage-public
refresh_token={{refresh_token}}

Manipulation : Ajouter scope=hello.read.
```

OUTPUT ATTENDU

200, mêmes droits.

EXPLICATION

Le scope explicite est compris dans les droits d'origine. Il ne doit pas modifier le sujet ni l'audience de l'autorisation.

## F04 — R6 avec `scope=admin`.

INPUT / PRÉPARATION

```
POST {{as}}/oauth2/token
Authorization: No Auth
grant_type=refresh_token
client_id=portage-public
refresh_token={{refresh_token}}

Manipulation : Ajouter scope=admin ; après le refus, retirer ce champ et réutiliser le token
initial.
```

OUTPUT ATTENDU

400 / invalid_scope ; le token initial reste utilisable avec le bon scope.

EXPLICATION

L'élévation de privilèges est refusée sans consommer le refresh token valide. La deuxième demande avec les droits permis doit réussir.

## F05 — Token de `portage-public` présenté par `demo-client` avec son bon Basic.

INPUT / PRÉPARATION

```
POST {{as}}/oauth2/token
Authorization: No Auth
grant_type=refresh_token
client_id=portage-public
refresh_token={{refresh_token}}

Manipulation : Token émis pour portage-public ; client_id=demo-client ; Basic de demo-client.
 Puis réessayer avec le propriétaire.
```

OUTPUT ATTENDU

400 / invalid_grant ; le propriétaire peut encore utiliser son token.

EXPLICATION

Le token est lié au registeredClientId stocké. Une tentative d'un autre client ne doit ni obtenir les droits ni détruire la famille du propriétaire.

## F06 — Token de `demo-client` présenté par `portage-public`.

INPUT / PRÉPARATION

```
POST {{as}}/oauth2/token
Authorization: No Auth
grant_type=refresh_token
client_id=portage-public
refresh_token={{refresh_token}}

Manipulation : Token émis pour demo-client ; No Auth et client_id=portage-public.
```

OUTPUT ATTENDU

400 / invalid_grant ; pas d'accès à l'autre utilisateur/client.

EXPLICATION

Le client public ne peut pas utiliser le grant d'un client confidentiel, même en possédant sa valeur de refresh token.

## F07 — R6 avec token inventé ou tronqué.

INPUT / PRÉPARATION

```
POST {{as}}/oauth2/token
Authorization: No Auth
grant_type=refresh_token
client_id=portage-public
refresh_token={{refresh_token}}

Manipulation : Essai refresh_token=invente ; essai avec token réel tronqué.
```

OUTPUT ATTENDU

400 / invalid_grant.

EXPLICATION

Le refresh token est une valeur opaque aléatoire recherchée côté serveur. Il ne peut pas être reconstruit à partir du sujet utilisateur.

## F08 — Client public avec `client_id` absent, vide ou inconnu.

INPUT / PRÉPARATION

```
POST {{as}}/oauth2/token
Authorization: No Auth
grant_type=refresh_token
client_id=portage-public
refresh_token={{refresh_token}}

Manipulation : Retirer client_id ; puis essayer client_id vide ; puis client_id=inconnu.
```

OUTPUT ATTENDU

Refus client OAuth, aucun token.

EXPLICATION

Le renouvellement public nécessite une identification explicite d'un client enregistré. La possession seule d'un token sans client ne suffit pas.

## F09 — Client confidentiel avec secret incorrect.

INPUT / PRÉPARATION

```
POST {{as}}/oauth2/token
Authorization: No Auth
grant_type=refresh_token
client_id=portage-public
refresh_token={{refresh_token}}

Manipulation : Famille de demo-client ; Basic demo-client avec secret incorrect.
```

OUTPUT ATTENDU

Refus client OAuth, généralement 401 / invalid_client.

EXPLICATION

L'authentification du client confidentiel reste obligatoire lors de chaque renouvellement, et pas uniquement au premier échange.

## F10 — Comparer les JWT avant et après R6.

INPUT / PRÉPARATION

```
POST {{as}}/oauth2/token
Authorization: No Auth
grant_type=refresh_token
client_id=portage-public
refresh_token={{refresh_token}}

Manipulation : Conserver localement le JWT avant et après renouvellement ; décoder les deux
sans les publier.
```

OUTPUT ATTENDU

Même sub, aud et scope ; nouvelles dates d'émission/expiration et nouveau jti.

EXPLICATION

sub, aud et scope sont conservés ; les dates et jti identifient une nouvelle émission. Le décodage seul ne vérifie pas la signature.

## S01 — R6 : A devient B ; rejouer A avec son client d'origine.

INPUT / PRÉPARATION

```
POST {{as}}/oauth2/token
Authorization: No Auth
grant_type=refresh_token
client_id=portage-public
refresh_token={{refresh_token}}

Manipulation : A -> B par un renouvellement valide ; envoyer ensuite refresh_token=A avec le
client d’origine.
```

OUTPUT ATTENDU

400 / invalid_grant.

EXPLICATION

L’historique conserve l’empreinte de A. Ce rejeu est interprété comme une compromission possible et révoque la famille.

## S02 — Après S01, utiliser B.

INPUT / PRÉPARATION

```
POST {{as}}/oauth2/token
Authorization: No Auth
grant_type=refresh_token
client_id=portage-public
refresh_token={{refresh_token}}

Manipulation : Après S01, envoyer refresh_token=B.
```

OUTPUT ATTENDU

400 / invalid_grant : toute la famille est invalidée.

EXPLICATION

Refuser seulement A serait insuffisant : un attaquant pourrait avoir obtenu B. Le dernier token de la famille doit aussi être invalidé.

## S03 — Après S01, obtenir une nouvelle autorisation C et renouveler C.

INPUT / PRÉPARATION

```
POST {{as}}/oauth2/token
Authorization: No Auth
grant_type=refresh_token
client_id=portage-public
refresh_token={{refresh_token}}

Manipulation : Après S01, refaire R3/R4 pour créer une famille C ; envoyer refresh_token=C.
```

OUTPUT ATTENDU

200 : une autre famille reste indépendante.

EXPLICATION

Une nouvelle autorisation crée une autre famille. La détection d'un rejeu ne doit pas bannir toutes les futures autorisations du client.

## S04 — A devient B ; présenter A avec l'autre client, puis B avec le bon client.

INPUT / PRÉPARATION

```
POST {{as}}/oauth2/token
Authorization: No Auth
grant_type=refresh_token
client_id=portage-public
refresh_token={{refresh_token}}

Manipulation : A -> B pour portage-public ; présenter A avec demo-client + Basic valide ;
présenter B avec portage-public.
```

OUTPUT ATTENDU

Mauvais client refusé ; B fonctionne, sans révocation de sa famille.

EXPLICATION

La détection de rejeu n'invalide une famille que si la demande identifie son client d'origine. Cette règle limite la révocation malveillante par un autre client.

## S05 — R7 sur le refresh token actif, puis R6.

INPUT / PRÉPARATION

```
POST {{as}}/oauth2/revoke
Authorization: No Auth
client_id=portage-public
token={{refresh_token}}
token_type_hint=refresh_token

Manipulation : Révoquer le refresh actif ; puis R6 avec exactement la valeur révoquée.
```

OUTPUT ATTENDU

Révocation 200, renouvellement 400 / invalid_grant.

EXPLICATION

La révocation doit persister dans l'autorisation Spring et être appliquée dès la tentative suivante de renouvellement.

## S06 — A devient B ; R7 sur A avec le bon client, puis R6 sur B.

INPUT / PRÉPARATION

```
POST {{as}}/oauth2/revoke
Authorization: No Auth
client_id=portage-public
token={{refresh_token}}
token_type_hint=refresh_token

Manipulation : A -> B ; token=A dans la révocation avec le bon client ; ensuite R6 avec B.
```

OUTPUT ATTENDU

Révocation 200 ; B refusé.

EXPLICATION

Un ancien token permet de retrouver la famille via l'historique d'empreintes ; la révocation doit atteindre son token courant.

## S07 — R7 avec token inconnu ou déjà révoqué.

INPUT / PRÉPARATION

```
POST {{as}}/oauth2/revoke
Authorization: No Auth
client_id=portage-public
token={{refresh_token}}
token_type_hint=refresh_token

Manipulation : token=inconnu ; puis répéter une révocation déjà effectuée.
```

OUTPUT ATTENDU

200, sans révéler l'existence du token.

EXPLICATION

Le protocole de révocation retourne 200 sans révéler si une valeur existe. Le critère d'efficacité est le refus d'un renouvellement ultérieur.

## S08 — Révoquer un token d'un autre client avec un client valide, puis utiliser le token avec son propriétaire.

INPUT / PRÉPARATION

```
POST {{as}}/oauth2/revoke
Authorization: No Auth
client_id=portage-public
token={{refresh_token}}
token_type_hint=refresh_token

Manipulation : Token actif de portage-public ; révoquer avec client_id=demo-client et son
Basic ; renouveler avec portage-public.
```

OUTPUT ATTENDU

Pas de révocation de l'autorisation du propriétaire.

EXPLICATION

Un client ne peut pas révoquer les tokens d'un autre. Observer le refus ou l'absence d'effet, puis vérifier que le propriétaire reste opérationnel.

## S09 — R7 puis R5 avec le JWT déjà émis.

INPUT / PRÉPARATION

```
GET {{rs}}/hello
Authorization: Bearer {{access_token}}

Manipulation : Conserver le JWT ; R7 sur le refresh ; appeler /hello avant puis après exp.
```

OUTPUT ATTENDU

Encore accepté avant expiration ; 401 après expiration.

EXPLICATION

Le RS valide localement un JWT autonome. La révocation des refresh tokens empêche une nouvelle émission, mais ne fournit pas de révocation immédiate du JWT déjà signé.

## S10 — Deux autorisations indépendantes ; révoquer seulement la première.

INPUT / PRÉPARATION

```
POST {{as}}/oauth2/token
Authorization: No Auth
grant_type=refresh_token
client_id=portage-public
refresh_token={{refresh_token}}

Manipulation : Créer deux familles A et X par deux R3/R4 ; révoquer A ; envoyer
refresh_token=X.
```

OUTPUT ATTENDU

Le renouvellement de la deuxième réussit.

EXPLICATION

L'état de révocation doit être limité au grant visé, même lorsque deux autorisations appartiennent au même client et au même utilisateur.

## P01 — Obtenir un refresh token ; arrêter puis relancer l'AS avec le même répertoire, la même clé et la même configuration ; R6.

INPUT / PRÉPARATION

```
POST {{as}}/oauth2/token
Authorization: No Auth
grant_type=refresh_token
client_id=portage-public
refresh_token={{refresh_token}}

Manipulation : Conserver le token ; arrêter et relancer seulement l'AS, avec même clé,
répertoire et configuration ; renouveler.
```

OUTPUT ATTENDU

200.

EXPLICATION

Les autorisations sont en base et non dans une collection mémoire. Ne pas effacer le volume ou changer d'identité de client pendant le test.

## P02 — A devient B ; redémarrer ; rejouer A puis présenter B.

INPUT / PRÉPARATION

```
POST {{as}}/oauth2/token
Authorization: No Auth
grant_type=refresh_token
client_id=portage-public
refresh_token={{refresh_token}}

Manipulation : A -> B ; redémarrer l’AS ; envoyer A, puis B, avec le bon client.
```

OUTPUT ATTENDU

Deux refus invalid_grant : l'historique survit au redémarrage.

EXPLICATION

La persistance du token courant seule ne suffit pas. L’historique de rejeu doit lui aussi survivre au redémarrage et permettre d’invalider B.

## P03 — Révoquer, redémarrer, tenter le renouvellement.

INPUT / PRÉPARATION

```
POST {{as}}/oauth2/token
Authorization: No Auth
grant_type=refresh_token
client_id=portage-public
refresh_token={{refresh_token}}

Manipulation : Révoquer le token ; redémarrer l’AS ; réutiliser la même valeur.
```

OUTPUT ATTENDU

Token toujours refusé.

EXPLICATION

Une révocation ne doit pas disparaître avec la JVM. Le token doit rester inutilisable après rechargement.

## P04 — Se connecter, redémarrer l'AS rapidement, conserver le cookie Postman ; appeler R3.

INPUT / PRÉPARATION

```
GET {{as}}/oauth2/authorize
Accept: text/html ; cookie authentifié
response_type=code
client_id=portage-public
redirect_uri={{redirect_uri}}
scope=hello.read
state={{state}}
code_challenge={{code_challenge}}
code_challenge_method=S256

Manipulation : Se connecter ; conserver le cookie ; redémarrer rapidement l’AS ; demander un
nouveau code.
```

OUTPUT ATTENDU

Code obtenu sans nouveau mot de passe, tant que la session n'a pas expiré.

EXPLICATION

Spring Session JDBC conserve le contexte de sécurité. La même session encore valide doit éviter une nouvelle saisie du mot de passe.

## P05 — Après connexion, ne plus utiliser la session pendant plus de 30 minutes ; appeler R3.

INPUT / PRÉPARATION

```
GET {{as}}/oauth2/authorize
Accept: text/html ; cookie authentifié
response_type=code
client_id=portage-public
redirect_uri={{redirect_uri}}
scope=hello.read
state={{state}}
code_challenge={{code_challenge}}
code_challenge_method=S256

Manipulation : Se connecter ; ne plus utiliser la session pendant au moins 30 min 10 s ;
demander un code.
```

OUTPUT ATTENDU

Nouvelle connexion nécessaire ; ne pas déduire une révocation des refresh tokens.

EXPLICATION

L’expiration de la session HTTP exige une nouvelle connexion. Elle est indépendante de la durée de vie des refresh tokens déjà délivrés.

## P06 — Obtenir un refresh token, attendre plus de huit heures sans renouveler, R6.

INPUT / PRÉPARATION

```
POST {{as}}/oauth2/token
Authorization: No Auth
grant_type=refresh_token
client_id=portage-public
refresh_token={{refresh_token}}

Manipulation : Obtenir A ; attendre au moins 8 h et une marge, sans jamais renouveler A ;
envoyer A.
```

OUTPUT ATTENDU

400 / invalid_grant.

EXPLICATION

Chaque refresh token a sa propre échéance. Un token expiré doit être rejeté même si sa famille n'a pas encore atteint sept jours.

## P07 — Maintenir une famille en la renouvelant avant chaque délai de huit heures, jusqu'à dépasser sept jours.

INPUT / PRÉPARATION

```
POST {{as}}/oauth2/token
Authorization: No Auth
grant_type=refresh_token
client_id=portage-public
refresh_token={{refresh_token}}

Manipulation : Maintenir une famille par des renouvellements espacés de moins de 8 h ;
essayer après 7 jours depuis la création.
```

OUTPUT ATTENDU

Plus de renouvellement après la limite absolue.

EXPLICATION

La limite absolue empêche une session OAuth renouvelable indéfiniment. Les dernières échéances de tokens sont plafonnées à cette limite.

## P08 — Arrêter l'AS ; tenter de rouvrir la même base avec une autre clé valide de 32 octets.

INPUT / PRÉPARATION

```
Hors HTTP / arrêt-démarrage de l’Authorization Server
AS arrêté ; même répertoire ; remplacer OAUTH_STORAGE_KEY par une autre clé Base64 valide de
32 octets ; démarrer.
```

OUTPUT ATTENDU

Démarrage refusé. Restaurer ensuite la bonne clé.

EXPLICATION

Le fichier chiffré ne doit pas être lu avec une autre clé. Le service doit refuser le démarrage ; restaurer ensuite la clé d’origine.

## P09 — Clé absente, Base64 invalide, ou clé décodée de 16 octets.

INPUT / PRÉPARATION

```
Hors HTTP / arrêt-démarrage de l’Authorization Server
Démarrer avec OAUTH_STORAGE_KEY absent ; puis Base64 invalide ; puis Base64 de 16 octets.
```

OUTPUT ATTENDU

Démarrage refusé.

EXPLICATION

La configuration exige une clé encodant 32 octets. Le service ne doit ni utiliser une clé de repli ni créer silencieusement un stockage non protégé.

## P10 — Vérifier les cookies de R1/R2.

INPUT / PRÉPARATION

```
GET {{as}}/login
Authorization: No Auth

Manipulation : Inspecter Set-Cookie après R1/R2 ; refaire sur une configuration HTTPS avec
Secure activé.
```

OUTPUT ATTENDU

HttpOnly et SameSite=Lax ; Secure lorsque la configuration TLS l'active.

EXPLICATION

HttpOnly limite l'accès JavaScript ; SameSite=Lax limite certains envois intersites ; Secure impose le transport HTTPS pour le cookie.

## P11 — Appeler l'ancien grant avec la configuration par défaut.

INPUT / PRÉPARATION

```
POST {{as}}/oauth2/token
Authorization: Basic demo-client / {{client_secret}}
grant_type=urn:portage:params:oauth:grant-type:subject
username={{username}}
password={{password}}
subject=subject-1
scope=hello.read
OAUTH_LEGACY_GRANT_ENABLED=false ; client demo-client avec Basic valide ; envoyer le grant
historique.
```

OUTPUT ATTENDU

Refus OAuth ; aucun token.

EXPLICATION

La compatibilité ne doit pas réintroduire implicitement un flux transmettant le mot de passe au client OAuth.

## P12 — Dans un environnement séparé, activer le grant historique et rejouer sa requête valide.

INPUT / PRÉPARATION

```
POST {{as}}/oauth2/token
Authorization: Basic demo-client / {{client_secret}}
grant_type=urn:portage:params:oauth:grant-type:subject
username={{username}}
password={{password}}
subject=subject-1
scope=hello.read
Environnement séparé ; OAUTH_LEGACY_GRANT_ENABLED=true ; redémarrer ; même demande historique
 valide.
```

OUTPUT ATTENDU

Access token seul, sans refresh token.

EXPLICATION

Le grant pédagogique peut encore servir aux anciens scénarios. Il émet seulement un access token, sans intégrer le cycle de refresh du nouveau flux.

# CON01 — Deux renouvellements concurrents

## Préparation / input

Créer une famille neuve pour portage-public. Copier son refresh token A dans concurrent_refresh_token. Envoyer une requête GET {{as}}/oauth2/jwks dont le Post-response lance les deux POST ci-dessous de manière asynchrone. Le script complet figure en annexe.

```
Deux requêtes indépendantes, avec la MÊME valeur A :
POST {{as}}/oauth2/token
Authorization: No Auth
grant_type=refresh_token
client_id=portage-public
refresh_token={{concurrent_refresh_token}}
```

## Output attendu

```
Réponse 1 (ordre non garanti) : 200 + JWT + refresh token B
Réponse 2 : 400 {"error":"invalid_grant"}
Puis POST refresh_token=B : 400 {"error":"invalid_grant"}
```

## Explication

Le verrou JDBC sérialise lecture, validation et sauvegarde. Une seule demande consomme A avec succès. La suivante retrouve l'empreinte d'un ancien token et invalide sa famille : B devient donc lui aussi inutilisable.

Deux clics ou deux appels asynchrones Postman ne garantissent pas une arrivée simultanée au serveur. Ils permettent de contrôler le résultat fonctionnel ; le test Java avec barrière de départ complète la preuve de concurrence. Un Runner exécutant deux requêtes successives ne prouve pas le chevauchement.

# 11. Scripts Postman : capture et rotation

## R1 — Capturer le CSRF (Post-response)

```
const match = pm.response.text().match(/name="_csrf"[^>]*value="([^"]+)"/);
pm.test("CSRF disponible", () => {
  pm.response.to.have.status(200);
  pm.expect(match).not.to.eql(null);
});
if (match) pm.environment.set("csrf", match[1]);
```

## R3 — Préparer state (Pre-request)

```
pm.environment.set("state", pm.variables.replaceIn("{{$guid}}"));
pm.environment.unset("authorization_code");
```

## R3 — Capturer le code et vérifier state (Post-response)

```
const location = pm.response.headers.get("Location") || "";
const code = location.match(/[?&]code=([^&]+)/);
const state = location.match(/[?&]state=([^&]+)/);
pm.test("Code et state valides", () => {
  pm.response.to.have.status(302);
  pm.expect(code).not.to.eql(null);
  pm.expect(state).not.to.eql(null);
  pm.expect(decodeURIComponent(state[1]))
    .to.eql(pm.environment.get("state"));
});
if (code) pm.environment.set("authorization_code", decodeURIComponent(code[1]));
```

# 12. Scripts Postman : tokens

## R4 et R6 — Capturer les tokens après réussite seulement

```
if (pm.response.code === 200) {
  const body = pm.response.json();
  pm.test("Réponse token nominale", () => {
    pm.expect(body.access_token).to.be.a("string").and.not.empty;
    pm.expect(body.refresh_token).to.be.a("string").and.not.empty;
    pm.expect(body.token_type.toLowerCase()).to.eql("bearer");
    pm.expect(body.scope).to.eql("hello.read");
    pm.expect(body.expires_in).to.be.within(298, 300);
    pm.expect(pm.response.headers.get("Cache-Control")).to.include("no-store");
  });
  pm.environment.set("access_token", body.access_token);
  pm.environment.set("refresh_token", body.refresh_token);
}
```

## R6 — Avant le renouvellement (Pre-request)

```
pm.environment.set("previous_refresh_token", pm.environment.get("refresh_token"));
```

## R6 — Vérifier la rotation (Post-response)

```
if (pm.response.code === 200) {
  pm.test("Refresh token remplacé", () => {
    pm.expect(pm.response.json().refresh_token)
      .not.to.eql(pm.environment.get("previous_refresh_token"));
  });
}
```

Pour tester un rejeu, utiliser une copie sans script de capture et choisir explicitement previous_refresh_token. Ne pas remplacer accidentellement la valeur active par une ancienne valeur.

# 13. Script Postman : concurrence

```
const token = pm.environment.get('concurrent_refresh_token');
const responses = [];
for (let i = 0; i < 2; i++) {
  pm.sendRequest({
    url: pm.environment.get('as') + '/oauth2/token',
    method: 'POST',
    header: {'Content-Type': 'application/x-www-form-urlencoded'},
    body: {
      mode: 'urlencoded',
      urlencoded: [
        {key: 'grant_type', value: 'refresh_token'},
        {key: 'client_id', value: 'portage-public'},
        {key: 'refresh_token', value: token}
      ]
    }
  }, (error, response) => {
    pm.test('Appel concurrent reçu', () => pm.expect(error).to.eql(null));
    if (error) return;
    responses.push(response);
    if (responses.length === 2) {
      pm.test('Une réussite et un refus', () => {
        pm.expect(responses.map(r => r.code).sort()).to.eql([200, 400]);
        pm.expect(responses.find(r => r.code === 400).json().error).to.eql('invalid_grant');
      });
      const winner = responses.find(r => r.code === 200);
      if (winner) pm.environment.set('refresh_after_race', winner.json().refresh_token);
    }
  });
}
```

Après les deux réponses, envoyer R6 avec refresh_token={{refresh_after_race}}. Attendu : 400 invalid_grant. Aucun token ni cookie ne doit être imprimé dans la console ou exporté dans un compte rendu.

# 14. Recette longue et compte rendu

## Expirations et redémarrages

Les scénarios P06 et P07 demandent des délais réels de huit heures et sept jours. Pour une recette accélérée, utiliser une construction de test séparée : modifier les TTL dans AuthorizationSecurityConfiguration et la limite de famille à ses deux emplacements dans RotatingRefreshTokenGenerator et RotatingAuthorizationService ; repartir d'une base de test neuve. Cette adaptation n'est pas fournie sous forme d'un paramètre d'environnement.

P08/P09 sont des tests de démarrage. Arrêter l'AS avant de changer sa clé, conserver le fichier existant et restaurer la bonne clé après le refus. Postman seul ne permet pas de prouver le chiffrement au repos : l'inspection du fichier et le test de réouverture avec mauvaise clé apportent une preuve distincte.

## Compte rendu sans secrets

| Champ | À renseigner |
|---|---|
| Scénario / date | Identifiant, horodatage et version du code |
| Contexte | Client, utilisateur, famille désignée par un nom fictif |
| Input | Requête et modifications, sans valeur réelle de token ou mot de passe |
| Output observé | Statut HTTP, error OAuth, présence/absence de champs |
| Verdict | Conforme / non conforme / non exécuté ; justification |
| Preuve | Capture assainie ou assertion Postman ; jamais un export de secrets |

La collection postman/pkce.collection.json couvre seulement échange, rotation, rejeu et refus après rejeu. Ses variables _first_refresh et _latest_refresh sont locales à un même run. Pour des clics Send indépendants, employer les variables d'environnement du présent dossier.

# 15. Sources et traçabilité

docs/AUTHORIZATION_CODE_PKCE.md

docs/PLAN_TEST_MANUEL_POSTMAN_PKCE.md

docs/KEY_SERVICE.md

docs/evidence/coverage.json

authorization-server/src/main/java/org/portage/authorizationserver/config/AuthorizationSecurityConfiguration.java

authorization-server/src/main/java/org/portage/authorizationserver/config/AuthorizationPersistenceConfiguration.java

authorization-server/src/main/java/org/portage/authorizationserver/security/RotatingAuthorizationService.java

authorization-server/src/main/java/org/portage/authorizationserver/security/RotatingRefreshTokenGenerator.java

authorization-server/src/main/java/org/portage/authorizationserver/security/PublicRefreshClientAuthentication.java

resource-server/src/main/java/org/portage/resourceserver/web/HelloController.java

## Références externes

https://www.rfc-editor.org/rfc/rfc7636.html#appendix-B

https://learning.postman.com/docs/use/send-requests/create-requests/request-settings

https://learning.postman.com/v11/docs/use/send-requests/response-data/cookies

https://learning.postman.com/latest-v-12/docs/tests-and-scripts/write-scripts/postman-sandbox-reference/pm-send-request

Les documents ARCHITECTURE.md et certaines preuves historiques décrivent une étape antérieure. Le présent dossier privilégie les classes courantes, les trois services et le flux PKCE. Le PDF est généré par scripts/build-architecture-pdf.py.